UDB VIRTUAL
Universidad Don Bosco, El Salvador

Desafío 1

Diseño y Programación de Software Multiplataforma

Carrera:

Ingeniería en Ciencias de la Computación

Grupo:

DPS941 G02T

Apellidos	Nombres	Carnet
Ramírez Rivas	Alberto Alkindi	RR160517

Fecha de entrega:

08 de marzo de 2026

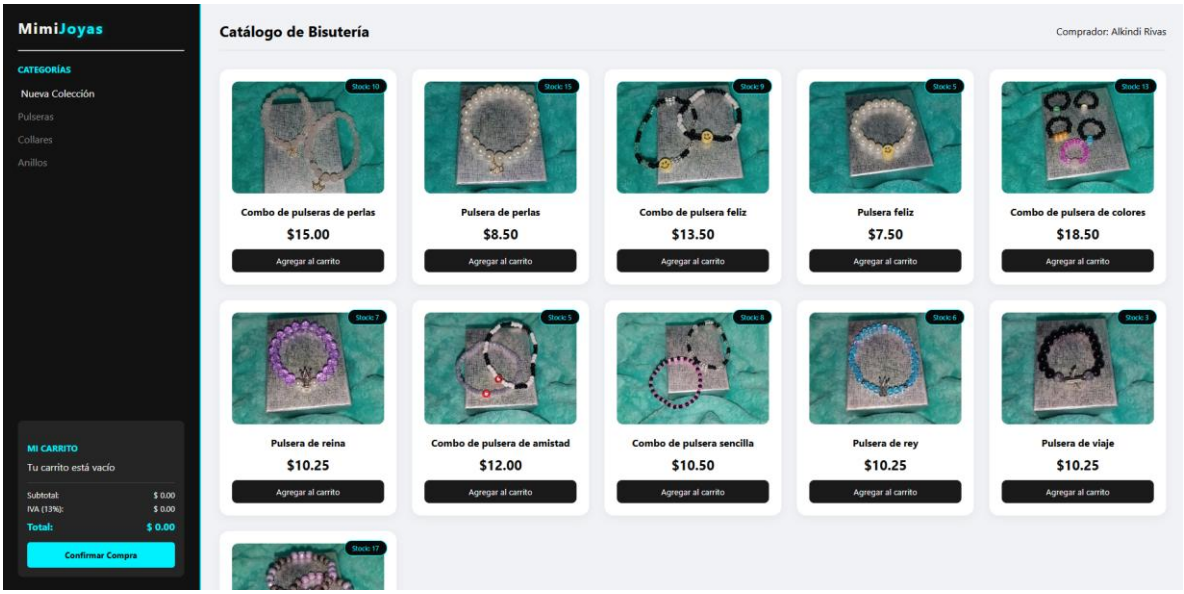
INTRODUCCIÓN

El presente trabajo se realiza el desarrollo de un sistema de carrito de compras de una tienda de bisutería y joyas. Aplicando los conceptos claves de JavaScript estudiado previamente en las clases, manipulación del DOM, manejo de eventos y trabajo de funciones para la interacción completa de la compra de artículos de bisutería.

Presentación del sistema

Para acceder a la aplicación debe de ingresar a la siguiente url:

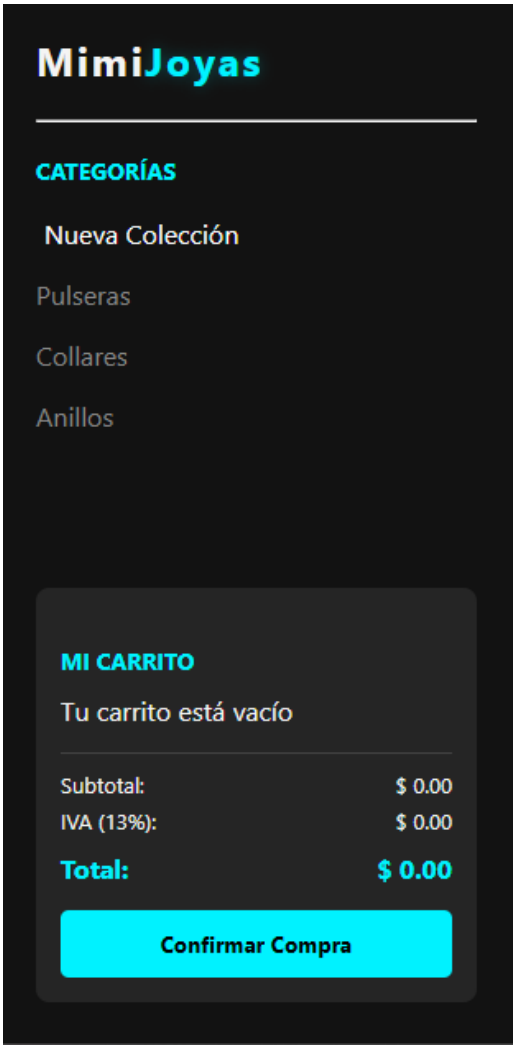
https://alkindi-rivas.github.io/DPS-Trabajo-de-investigac-n-1/



El sistema está alojado en Github Pages, además teniendo su propio repositorio público: https://github.com/alkindi-rivas/DPS-Trabajo-de-investigac-n-1

La aplicación está realizada en las tecnologías de HTML5, CSS3 y JavaScript.

Estructura de la aplicación



Menú lateral

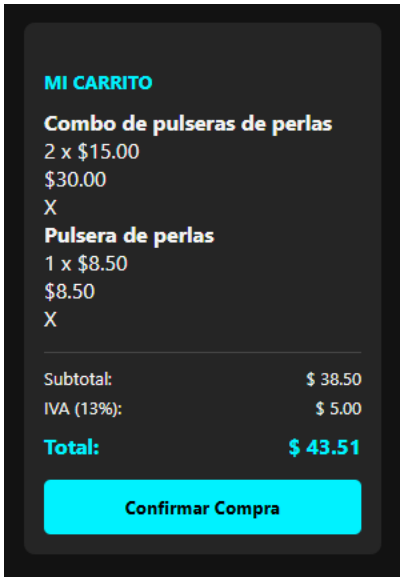
La aplicación posee un panel izquierdo en donde se muestran una representación de las categorías de los productos que se poseen (actualmente solo existe la categoría de nueva colección).

En la parte inferior posee el módulo del carrito de compras que se actualiza en tiempo real cuando el cliente agrega un producto.

El módulo del carrito de compras posee su propia validación, cuando se le da clic al botón "Confirmar Compra" sin antes haber seleccionado un producto, lanzará una alerta que el carrito está vacío y le indicará que primero hay que seleccionar un producto.

Actualización del carrito de compras

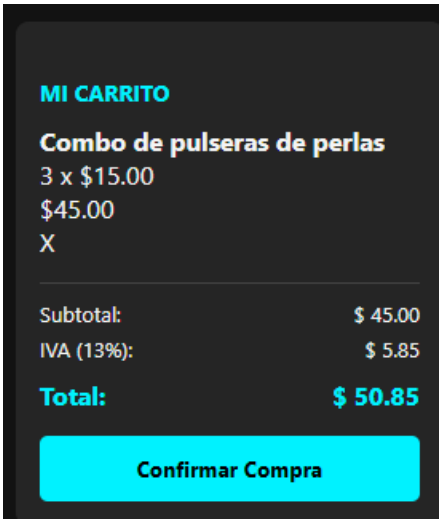
Cuando se añade un producto nuevo, la información del nombre del producto, cantidad, y precio total de la cantidad del producto y el precio total de compra, añadiendo su respectivo impuesto.



Proceso de compra

Para realizar el proceso de compra, debe de seleccionar primero un producto dando clic en “Agregar al carrito”. Se debe de tener en cuenta que cada producto posee su respectivo stock, en dado caso el stock del producto llegue a 0, el producto aparecerá Agotado, y ya no podrá seleccionar dicho producto.

Una vez seleccionado el producto usted verá un resumen de la compra en su carrito, usted puede eliminar su producto seleccionado dando clic en la “X”. Cuando esté seguro de su compra, procedemos a dar clic en la opción “Confirmar Compra”.

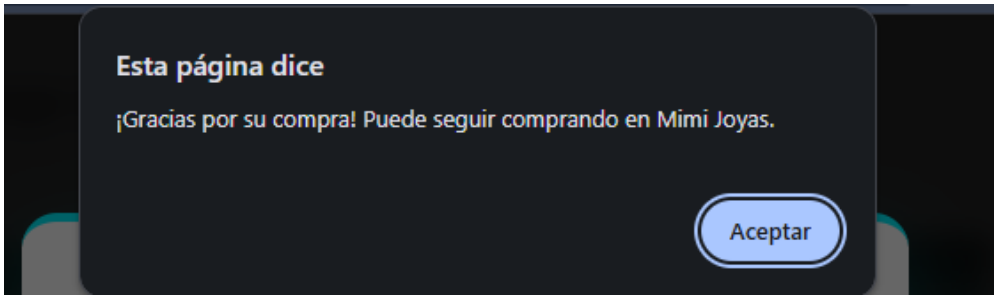


Luego le saldrá un resumen con la facturación creada dinámicamente.

Podrá observar:

- Nombre del producto comprado
- Cantidad del producto
- Precio Unitario del producto
- Subtotal de todos los productos.
- Impuestos aplicados
- Total, a pagar

Una vez corrobore sus datos de compra, podrá dar clic al botón “Nueva Compra” en donde le aparecerá una alerta agradeciendo su compra.



Si desea seguir comprando, lo podrá seguir haciendo, teniendo en cuenta que se actualiza el stock del producto en tiempo real y se irá descontando el stock a medida vaya realizando sus compras en la aplicación.

Backend de la aplicación

Definición de la clase producto. Dentro de la clase inicializo los atributos del objeto producto por medio del constructor e inicializo las variables, asignando el parámetro a cada atributo. (id, nombre, precio, stock e imagen)

```
// Definición de clase Producto
class Producto {
  // Constructor
  constructor(id, nombre, precio, stock, imagen) {
    this.id = id;
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
    this.imagen = imagen;
  }
```

Se creo una matriz en donde están almacenada la información de todos los productos para que se carguen automáticamente en la aplicación.

```
const inventario = [
  new Producto(1,"Combo de pulseras de perlas", 15, 10, "images/articulo_1.jpeg",),
  new Producto(2, "Pulsera de perlas", 8.5, 15, "images/articulo_2.jpeg"),
  new Producto(3, "Combo de pulsera feliz", 13.5, 9, "images/articulo_3.jpeg"),
  new Producto(4, "Pulsera feliz", 7.5, 5, "images/articulo_4.jpeg"),
  new Producto(5, "Combo de pulsera de colores", 18.5, 13, "images/articulo_5.jpeg"),
  new Producto(6, "Pulsera de reina", 10.25, 7, "images/articulo_6.jpeg"),
  new Producto(7, "Combo de pulsera de amistad", 12, 5, "images/articulo_7.jpeg"),
  new Producto(8, "Combo de pulsera sencilla", 10.5, 8, "images/articulo_8.jpeg"),
  new Producto(9, "Pulsera de rey", 10.25, 6, "images/articulo_9.jpeg"),
  new Producto(10, "Pulsera de viaje", 10.25, 3, "images/articulo_10.jpeg"),
  new Producto(11, "Combo de pulsera amatista", 13.5, 17, "images/articulo_11.jpeg"),
];
```

En la siguiente función se recorre la matriz declarada anteriormente por medio de un foreach y mostrando el código por medio de un innerHTML, para poder imprimir todos los datos de los productos en un contenedor html previamente creado y llamado por su ID.

```
// Funcion para mostrar los carritos en la vista del usuario

function mostrarProductos() {
  const contenedor = document.getElementById("lista-productos");
  contenedor.innerHTML = "";

  inventario.forEach((producto) => {
    contenedor.innerHTML += `
            <article class="product-card">
                <div class="badge-stock">Stock: ${producto.stock}</div>
                <img src="${producto.imagen}" alt="${producto.nombre}">
                <h4>${producto.nombre}</h4>
                <p class="product-price">$${producto.precio.toFixed(2)}</p>
                <button class="btn-add" onclick="agregarCarrito(${producto.id})"
                    ${producto.stock === 0 ? "disabled" : ""}>
                    ${producto.stock === 0 ? "Agotado" : "Agregar al carrito"}
                </button>
            </article>
        `;
  });
}

mostrarProductos();
```

Luego se posee una función para agregar todos los productos al carrito en donde se ocupa como parámetro el “id” que se posee de cada artículo. Se busca si el id se encuentra y realizo una condición en donde se verifica que el producto y su stock sean mayores a cero y si es el caso se irá restando el stock del producto cuando la persona seleccione el producto.

Posteriormente se crea otra condición en donde se evalúa que, si el producto está en el carrito, aumente en el carrito la cantidad a comprar de cada producto que se seleccione y se coloque la información de dichos productos.

```
// Funcion para agregar productos al carrito

function agregarCarrito(id){
    const producto = inventario.find(p => p.id === id); // Busqueda del produc

    // Control de stock

    if(producto && producto.stock > 0){
        producto.stock--; // Resta al stock del producto

        const productoEnCarrito = carrito.find(item => item.id === id);

        if(productoEnCarrito){
            productoEnCarrito.cantidad++; // Si el producto ya esta en el carr
        }else{
            carrito.push({
                id: producto.id,
                nombre: producto.nombre,
                precio: producto.precio,
                cantidad: 1
            })
        }

        actualizarInterfaz();
    } else {
        alert("Este artículo está agotado.")
    }
}
```

Luego existe la función para eliminar el producto del carrito, en donde siempre se toma como parámetro de la función el id, se hace primero la búsqueda que si o si debe de existir dicho id del producto y luego inicie la condición que el producto debe de ser estrictamente diferente a -1 para actualizar.

Luego localizo el producto en el inventario y busco el producto en el arreglo “inventario” y luego se accede a la propiedad “Stock” del producto encontrado y se suma la cantidad que el usuario tenía en el carrito de compras.

Se finaliza modificando el arreglo eliminando un producto en la posición especificada y se finaliza llamando a la función “actualizarInterfaz” para que vea los cambios al instante el usuario.

```
// Función para eliminar y actualizar datos

function eliminarCarrito(id){
    const index = carrito.findIndex(item => item.id === id);

    if (index !== -1){
        // Actualizo el producto cuando se elimina del carrito
        const cantidadOriginal = inventario.find(p => p.id === id);
        cantidadOriginal.stock += carrito[index].cantidad;

        carrito.splice(index, 1); // Vacio el carrito
        actualizarInterfaz();
    }
}
```

Para finalizar existe la función “actualizaraInterfaz”, en donde primero se manda a llamar la función “mostrarProductos” para mostrar en tiempo real el catálogo de los productos

Luego se llamada por el elemento ID de html el contenedor donde estarán los datos del carrito de compra.

Se realiza un foreach para que todos los productos seleccionados para comprar se muestren en el carrito de compras en la interfaz del usuario por medio de un innerHTML, colocando el nombre, cantidad, precio del/los productos y el total calculado con sus respectivos impuestos.

Se calcula el IVA más el subtotal de los productos del carrito y se muestran en el elemento HTML con el ID “tax-val”.

```
function actualizarInterfaz(){

    mostrarProductos(); // Actualizo el catalogo de productos

    const listaCarrito = document.getElementById('cart-items-container');
    listaCarrito.innerHTML = '';

    let subtotal = 0;

    // For each para que cada producto se imprima en el carrito
    carrito.forEach(item =>{
        const totalPorProducto = item.precio * item.cantidad;
        subtotal += totalPorProducto;

        listaCarrito.innerHTML += `
            <div class="cart-item">
                <div>
                    <strong>${item.nombre}</strong><br>
                    ${item.cantidad} x $${item.precio.toFixed(2)}
                </div>
                <div class="aqua-text">$${totalPorProducto.toFixed(2)}</div>
                <span class="btn-remove" onclick="eliminarCarrito(${item.id})">X</span>
            </div>
        `;
    })

    // Calculo de IVA y total de compra
    const iva = subtotal * 0.13;
    const total = subtotal + iva;

    document.getElementById('subtotal-val').innerText = subtotal.toFixed(2);

    if(document.getElementById('tax-val')) document.getElementById('tax-val').innerText = iva.toFixed(2);
    document.getElementById('price-total').innerText = total.toFixed(2);
}
```

La siguiente función muestra el como se imprime el carrito de compras y se valida que si el array del carrito es estrictamente igual a cero, se muestra una alerta al usuario.

```
function procesarCompra(){
    if(carrito.length == 0){
        alert("El carrito está vacio. Primero selecciona un producto.")
        return;
    }

    const seccionFactura = document.getElementById('factura-section');
    const detalleFactura = document.getElementById('factura-detalle');

    seccionFactura.style.display = 'flex';

    // Creando estructura de factura
    let tablaHtml = `
        <table style="width:100%; border-collapse: collapse; margin-top: 20px;">
            <thead>
                <tr style="border-bottom: 2px solid var(--aqua); text-align: left;">
                    <th style="padding: 10px;">Producto</th>
                    <th>Cant.</th>
                    <th>P. Unit</th>
                    <th>Subtotal</th>
                </tr>
            </thead>
            <tbody>
    `;

    let subtotalFinal = 0;
```

Luego realizo un ciclo foreach, creo una variable que poseerá el total del producto calculado por el precio del producto más la cantidad que hay en el carrito de compras. El subtotal del producto irá incrementando por la cantidad que se seleccione.

Luego se imprimen los resultados mostrando, la cantidad, precio y total de cada producto mostrado a dos decimales.

```
    let subtotalFinal = 0;

    // Recorriendo cada producto con su informacion
    carrito.forEach(item => {
        const totalProducto = item.precio * item.cantidad;
        subtotalFinal += totalProducto;
        tablaHtml += `
            <tr style="border-bottom: 1px solid #eee;">
                <td style="padding: 10px;">${item.nombre}</td>
                <td>${item.cantidad}</td>
                <td>$${item.precio.toFixed(2)}</td>
                <td>$${totalProducto.toFixed(2)}</td>
            </tr>
        `;
    });

    // Mostrando impuestos + total de factura
    const impuesto = subtotalFinal * 0.13;
    const totalGeneral = subtotalFinal + impuesto;
    tablaHtml += `
            </tbody>
        </table>
        <div style="margin-top: 20px; text-align: right; line-height: 1.8;">
            <p>Subtotal: <strong>$${subtotalFinal.toFixed(2)}</strong></p>
            <p>IVA (13%): <strong>$${impuesto.toFixed(2)}</strong></p>
            <h3 class="aqua-text" style="font-size: 24px;">Total a Pagar: $${totalGeneral.toFixed(2)}</h3>
        </div>
    `;

    detalleFactura.innerHTML = tablaHtml;
}
```

En la captura anterior se calcula el impuesto y el subtotal de la compra global y se imprime redondeando a dos decimales.

Por último, se creó la función “nuevaCompra” en donde reiniciamos el array del carrito, dejándolo vació, mandamos a llamar la función “actualizarInterfaz” que nos permite vaciar el carrito. Al finalizar el proceso el usuario ve una alerta agradeciendo la compra y permitiendo realizar una nueva compra, con el stock actualizado en tiempo real.

```
// Funcion para que el usuario siga comprando

function nuevaCompra(){
    carrito = []; // Vacio el carrito para una nueva compra

    document.getElementById('factura-section').style.display = 'none';
    actualizarInterfaz();

    alert("¡Gracias por su compra! Puede seguir comprando en Mimi Joyas.");
}
```